Listing 11.8 Mock service request listener

```
val provider = System.getenv()
            .getOrDefault("PACT_PROVIDER", "BiWiremockMoviesAPI")
        wireMockServer!!.addMockServiceRequestListener(
            WireMockPactGenerator
                .builder("BiMoviesAndroidApp", provider)
                .build()
        )
```

With the server and listener setup, include the mock for the “movies” URL within the test, as shown in listing 11.9. Chain the `get` method to the instance of the WireMock server, which contains the request URL to intercept and respond to headers, status code, and body.

Listing 11.9 WireMock mock response

```
wireMockServer?.get {
            url equalTo "/movies"
        }!! returns {
            header = "Content-Type" to "application/json"
            statusCode = 200
            body = """{
            "data": [{
              "Id": 100,
              "Name": "Frozen",
              "Year": 2013
            }]
            }
            """
        }
```

Once the mock has been set up, the client request and response assertion can be made within the test, as follows.

Listing 11.10 JUnit test request and assertion

```
val movies = runBlocking { MovieClient(url).getMovies()};

assertThat(movies.data[0].Id, equalTo(100))
```

Running the test will generate the Pact contract when the test passes successfully (code in the GitHub repository can be found at consumer-mobile/app/src/test/java/com/ example/movie/MoviesApiBiTest.kt). The Pact file can be found in the app/build/pacts folder, as follows.